Lecture Three

Study handout — text extracted from the original slides (wording preserved). Logic symbols fixed (∧ ∨ ¬ ⇒ ⇐ ≡ ≠ ⊥). About half the paper or less. 78 slides. Black text on white.

How to use: skim slide titles, study bullets in order, read formulas as written.

Slide 1 — Department of Computer Sciences

(No body text on this slide.)

Slide 2

أهداف الكلية رسالة الكلية الكلية
رؤية
عملعداد خريج متميز قادر على المنافسة في سوق الإ
ه 1.
اعتماد طرق تدريس متميزة وبرامج تعليمي
احليمجالها مي ن تصبح رائدة فأب ة
تلتزم الكلي
والقدرة على العمل مع وعالية الجودة وتتماشى مع متطلباتة
شامل
ا من خالل تقديمها لبرامج أكاديمية
وعالمي
.
االخرين من التخصصات االخرى
المجتمع وسوق العمل
رتوجيه االبحاث التطبيقية لتخدم أغراض رؤية مص
2. من خالل خلق بيئة
وإجراء البحوث اإلبداعية
متميزة في الدراسات العلياة.
ومهني
.
0302 تمكن من االستغالل
إدارية وأكاديمية محفزة
تي
استمرار الكلية في تطوير وإدارة البرامج ال
بناء شراكة مجتمعية فاعلة مع القطاعين العام
3. مثل للموارد والتكنولوجيا الحديثة
أل .
والخاص والدولي
بناء الشراكة مع المجتمع المحلي
تخدم احتياجات سوق العمل والباحثين
تحقيق تميز في أألداء االألكاديمي واإلداري والمهني.
ن 4.
طرح دورات تدريبية وبرامج مهنيه للعاملي
.
والمتخصصين
التوسع في تطبيق معايير الجودة الشاملة.
.5
في المنظمات المختلفة
استحداث برامج دراسية تتماشى مع رؤية مصر
6. اإلدارية كجزء من نشاط
تدريب الكوادرجيين
التحسين المستمر من خالل تثقيف الخري
.
2030
خدمات المجتمع وتعزيز الفضول الفكريمع
وترسيخ مبادئ التعلم مدى الحياة لديهم
تطوير الخدمات المجتمعية والبيئية.
.7
والمرونة للنمو اجات
توفير البنية التحتية التي توفر احتي
توبالتوسع في قبول الطالب الوافدين وتحسين مس

8. التكنولوجيا واألساليب الجديدة في
تطوير
الطالب وتحديثها بما يضمن تميزها وتميز
.
الخدمة المقدمة لهم
اإلحصاء وعلوم الحاسب ونظم وتكنولوجيا ا
خريجيه
تحسين البنيـة التحتية للكلية.
.9
المعلومات
سينتنمية المهارات اإلبداعية والتفكير النقدي للدار.
.10

Slide 3

Faculty of Graduate Studies for Statistical Research Department of Computer Sciences Course title: Advanced Topic in Software Engineering Course Code: CS 601 Dr. Fatma A. El -Licy

Slide 4 — Lecture Three.P1

(No body text on this slide.)

Slide 5 — AGENDA

• Data Structure
• Bunch
• Set
• String
• List
• Function Theory

Slide 6 — Data Structure 24 October 2026

(No body text on this slide.)

Slide 7 — Data Structure

• Bunch unpackaged unindexed
• Set packaged unindexed
• String unpackaged indexed
• List packaged indexed 24 October 2026 7

Slide 8 — Bunch Theory 24 October 2026

(No body text on this slide.)

Slide 9 — BUNCH THEORY

• Abunch is a mathematical data structure used to represent a collection of things.
• Any number, character, or binary value (and later also set, string of elements, and list of elements) is an elementary bunch, or element.
0, 5, 3, 9 and A, 7, &, ⊥ are bunches
• Given the bunches A and B
A, B “ A union B ”
A ‘ B “ A intersection B ”
24 October 2026 9

Slide 10 — Bunch Theory

• Abunch is a mathematical data structure used to represent a collection of things.
¢A is “size of A ”, “cardinality of A”
¢2 = 1 ¢(0, 2, 5, 9, 9) = 4
• The Binary inclusion:
A:B “ A is in B” or “ A is included in B”
2: 0, 2, 5, 9 2: 2 9: 0, 5, 3, 9
A bunch consists of its elements (not contains its elements)
24 October 2026 10

Slide 11

Bunch Theory: AXIOMS Let x and y be two elements (elementary bunches), and
A, B, and C be arbitrary bunches, then x: y ≡ x=y elementary axiom x: A, B ≡ x: A ∨ x: B compound axiom
A, A ≡ A A ‘A ≡ A idempotence
A, B = B, A A ‘B ≡ B ’A symmetry
A, (B, C) ≡ (A, B), C A ‘(B ‘C) ≡ (A ‘B) ‘C associativity
A, B: C ≡ A: C ∧ B: C anti-distributivity
A: B ‘C ≡ A: B ∧ A: C distributivity 24 October 2026 11

Slide 12 — Bunch Theory: AXIOMS

A: A, B generalization A ‘B: A specialization
A: A reflexivity A: B ∧ B: A ≡ A=B anti-symmetry
A: B ∧ B: C ⇒ A: C transitivity Colon (inclusion of the Bunch) is ordering (reflex + anti-symmetry + trans)
Colon is the ordering relation on Bunch (like ≤ in numbers and ⇒ in Binary)
Remember
¢x = 1, bunch size
¢ (A, B) + ¢(A‘B) = ¢A + ¢B size (bunch is not idempotent)
¬ x: A ≡ ¢(A ‘x) = 0 size A: B ⇒ ¢A ≤ ¢B size 24 October 2026 12

Slide 13

Bunch Theory: USEFUL BUNCHES null the empty bunch bin = ⊤, ⊥ the binary values nat = 0, 1, 2,... int =..., –2, –1, 0, 1, 2,...
1/2 rat =..., –1, 0, 2/3,... real =..., 2,... the real numbers xnat = 0, 1, 2,.. ∞ +1 the extended natural numbers xint = –∞,.. –2, –1, 0, 1, 2,.. ∞ +1 the extended integer numbers xrat = –∞,..., –1, 0, 2/3,.. ∞ +1 the extended rational numbers xreal = –∞,.. ∞+1 the extended real numbers char =..., “a”, “A”,..”Z” the character values 24 October 2026

Slide 14 — Bunch Theory: NOTATIONS

• x,..y x ≤ y for integers up to infinity this is an axiom only for integer items/numerals
• i: x,..y ≡ x≤ i<y
• i:5,..19 ≡ 5 ≤ i< 19
• ¢(x,..y) ≡ y-x 0,..3 ≡ 0,1,2 0,..1 ≡ 0 0,..0 ≡ null 0,..∞ ≡ nat 0,..∞ + 1 ≡ xnat 24 October 2026 14

Slide 15 — Bunch Theory: Distribution Property

• Most operators distribute over bunch union.
• -null ≡ null
• – (A, B) ≡ –A, –B -(1, 4, 6, 9) ≡ -1, -4, -6, -9
A+ null ≡ null ≡ null + A null + 10 = null (A, B) + (C, D) ≡ A+ C, A+D, B+C, B+D (1, 2) + (10, 20) = 11, 12, 21, 22 (1,2) + 10 = 11, 12 1+ 10 = 11 nat + 2 ≡ 2, 3, 4, 5, 6,… nat × 2 ≡ 0, 2, 4, 6,…
2 nat nat ≡ 0, 1, 4, 9, 16,… 2 = 1, 2, 4, 8, …
24 October 2026 15

Slide 16 — DATA STRUCTURE

• Bunch unpackaged unindexed
• Set packaged unindexed 24 October 2026 16

Slide 17 — set Theory 24 October 2026

(No body text on this slide.)

Slide 18 — SET THEORY

• S={bunch} S={A} set containing A
• ~S content of a set (bunch)
S ={1, 4, 8} is a set then ~S =1, 4, 8 is a bunch
{1, 4, 8}, 9 a bunch of 2 elements
{{1, 4, 8}, 9} a set containing 2 elements a set and a number
• {null} the null set; {nat} the set of natural numbers
• {0,1,2} ≡ {0,..3}
• ${1, 4, 8} ≡ 3 size of the set (cardinality)
• ⊂ (0,1) = Power (0, 1) ≡ {null}, {0},{1},{0,1}

Slide 19 — SET THEORY

• Let S be a set, and let A and B be anything.
{A}
 A structure
• S={ 1,2,3,7} ≠ 1,2,3,7 yet ~S = 1,2,3,7 {~S}
≡ S set formation
• ~S = 1,2,3,7 then {~S }= {1,2,3,7} =S ~{A}
≡ A “contents”
${A} ≡ ¢A “size”, “cardinality”
• ${A }=${ 2, 3, 8, 9} = 4 = ¢A = ¢ (2,3,8,9) = 4 24 October 2026 19

Slide 20 — Set Theory

• Let S be a set, x be an element and let A and B be anything.
A ∈ {B} ≡ A: B “elements”; x ∈ {B} ≡ x: B
{A} {B} ≡ A: B “subset”
{A}: ⊂ B ≡ A: B “power ”
{A} {B} ≡ {A, B} “union”
{A} ∩ {B} ≡ {A ‘B} “intersection”
{A} = {B} ≡ A = B “equation”
24 October 2026 20

Slide 21 — DATA STRUCTURE

• Bunch unpackaged unindexed
• Set packaged unindexed
• String unpackaged indexed 24 October 2026 21

Slide 22 — String Theory 24 October 2026

(No body text on this slide.)

Slide 23 — STRING THEORY

• String are indexed sequences nil the empty string 8 a one-item string 4;5;8;9 a four-item string 4; 5; 8; 9 ↔ (4;5;8;9) = 4 string length 0 1 2 3 4 5 6 7 8 (4;5;8;9) = item in index 2 = 8
• At index n, the number of processed items (count) is n the next item to be processed is n 24 October 2026 23

Slide 24

String Theory (4;5;8;9) = 8;5;9 substring 2;1;3 Let n be an extended natural and S be a string, and i be an item
• n*S is a string composed of n copies of S joined together.
3 * (0; 1) ≡ 0; 1; 0; 1; 0; 1
• Without a left operand, *S means the bunch union of all strings formed by joining any number of copies of S.
*S ≡ nil, 1*S, 2*S, … (bunch union of strings)
*(0; 1) ≡ nil, 0; 1, 0;1;0;1,...
Now let n be an index of S and i be an item (not necessarily of S):
• S □ n□ i is a string like S except that the item at index n is i.
i.e., (store the item i in the index n)
(3; 5; 9)□ 1□ 8 ≡ 3; 8; 9

Slide 25 — String Theory

• 3; 6; 4; 7 < 3; 7; 2 (as if they were decimal numbers: 3647 < 3720)
• 3; 6; 4 < 3; 6; 4; 7 Lexicographic order.
• Let i be an item, S and T are strings, and n: xnat then are nil is the empty string, S;T is “ S join T ”
Strings
S is “ item(s) in index T of S”, n*S is “ n copies of S ”
T
S □ n□ i is “ S but at n there's i ”,
*S “copies of S ” is a bunch of strings,, not a string (Kleene star after the logician Stephen Kleene)
↔ S “length of S ” is an extended natural number.
24 October 2026 25

Slide 26

Axioms of String Theory Let S, T, and U be strings, i and j are items, and n is an extended natural number.
S; nil ≡ S ≡ nil; S identity
S; (T; U) ≡ (S; T); U associativity ↔ nil ≡ 0 base ↔ i ≡ 1 base ↔ (S; T) ≡ ↔ S + ↔ T idempotent
S ≡ nil ↔ S< ∞ ⇒ (S; i; T) ≡ i S ≡ S; S nil ↔ S T;U T U 24 October 2026 26

Slide 27

Application of String Axioms Let S =2; 3; 4; 5; 8; 9, T= 0; 2; 1, U = 4; 5 be strings and i = 10 is an item
• ↔(S; T) ≡ ↔S + ↔T ↔(S; T) ≡ ↔ (2; 3; 4; 5; 8; 9; 0; 2;1) = 9 ↔S + ↔T = 6 +3 = 9
• ↔S < ∞ ⇒ (S; i; T) ≡ i ↔ S 6 < ∞ ⇒ (2; 3; 4; 5; 8; 9; 10; 0; 2;1) ≡ 10
• S ≡ S; S S ≡ (2; 3; 4; 5; 8; 9) = 2; 4; 3; 8; 9
T;U T U T;U 0; 2; 1; 4; 5
S; S ≡ (2; 3; 4; 5; 8; 9); (2; 3; 4; 5; 8; 9)≡ (2; 4; 3); (8; 9)
T U 0;2;1 4;5 = 2; 4; 3; 8; 9 ≡ S T;U 24 October 2026 27

Slide 28

Axioms of String Theory Let S, T, and U be strings, i and j are items, and n: xnat.
S = (S; S) □ = (S) 0*S = nil (T;U) T U □ T U □
(n+ 1)*S = n*S; S ↔ S< ∞ ⇒ (S; i)□ ↔ S □ j ≡ S; j↔ S< ∞ ⇒ (S; i)□ ↔ S □ j; T≡ S; j; T↔ S< ∞ ⇒ nil ≤ S < S; i; T Order wise ↔ S< ∞ ⇒ (i<j ⇒ S; i; T < S; j; U) Order wise ↔ S< ∞ ⇒ (i=j ≡ S; i; T = S; j; T) Order wise 24 October 2026 28

Slide 29 — Axioms of String Theory

S, T and U be strings, i and j are items, and n:xnat.
 (x;..y) x ≤ y for integers up to infinity, this is an axiom for only integer items/numerals
 ↔ (x;..y)= y –x (string of integers starting at x up to but not included y)
 ↔ (x;..x+1) = x+1–x =1 x;..x+1 ≡ x ↔ (x;..x) ≡ 0 x;..x ≡ nil
 (x;..y); (y;..z) ≡ x;..z where x ≤ y ≤ z 24 October 2026 29

Slide 30 — Axioms of String Theory

θ “abcdefghij” = “def”
3;..6
θ String join distributes over bunch union:
let A= a, b, c B= r, s, t and D, E be bunch unions
• A; null; B ≡ null A; nil; B ≡ A; B
• (A, B); (D, E) why do we need parenthesis??
=A;D, A;E, B;D, B;E ≡ a;D, b;D, c;D, a;E, b;E, c;E, r;D, s;D,;D, r;E, s;E, t;E
θ YET A, B; D, E = A, B; D, E is union of a bunch union (A) and string (B;D) and a bunch union (E)
≡ a, b, c, (r, s, t); D, E = a,b,c, r;D, s;D, t;D, E = A, r;D, s;D, t;D, E 24 October 2026 30

Slide 31 — Axioms of String Theory

• ¢(E; F) ≤ ¢E × ¢F Let us examine this inequality Ex 1: ¢((A, B); (A, D)) let A= a, b, c B= r, s, t = ¢(A;A, A;D, B;A, B;D) = 4 ¢ (A,B) = 2 ¢ (A,D) =2
• ¢((A,B); (C,D)) = 4 ≤ ¢(A,B) × ¢(A,D) = 4
• (A, B); (D, E)
=A;D, A;E, B;D, B;E≡ a;D, b;D, c;D, a;E, b;E, c;E, r;D, s;D, t;D, r;E, s;E, t;E ¢ ((A,B); (D,E)) = 12
¢ (A,B) × ¢(D,E) = ¢ (a,b,c,r,s,t) × ¢(D,E) = 6X 2 = 12 24 October 2026 31

Slide 32 — ¢(E; F) ≤ ¢E × ¢F

• LET E= 1, 2 and F= 1, 2; 8 (string join(;) distribute over bunch union (,))
¢E = 2 ¢F= 2 ¢E × ¢F = 4
¢(E; F) = ¢ ((1,2); (1, 2;8)) == bunch length of String expression? not possible or may be an invalid operation?!!!!
Let us distribute string join over bunch to obtain a Bunch union ((1,2); (1, 2;8)) = (1;1, 1;2;8, 2;1, 2;2;8) is Bunch union of strings, then one can get its size
¢ (1;1, 1;2;8, 2;1, 2;2;8) == length of Bunch union == 4 ¢(E; F) ≤ ¢E × ¢F 4 ≤ 4 24 October 2026 32

Slide 33 — Axioms of String Theory

¢ nil =1 i.e., ¢ “” = 1 each is an elementary bunch
• Astring of bunches is equal to a bunch of strings, as seen in the previous examples.
• Also 0; 1; 2: nat; 1; (0,..10) because 0: nat and 1: 1 and 2: 0,..10 0; 1; 2: nat; 1; (0,..10): 3*nat: *nat 24 October 2026 33

Slide 34 — Axioms of String Theory

• Astring is an element (elementary bunch) just when all its items are elements 0; 1; 2 is a bunch element, but nat; 1; (0,..10) is not.
• The * operator distributes over bunch union in its left operand only.
null*A = null (A, B) * C = A*C, B*C *C = nat* C 24 October 2026 34

Slide 35 — Data Structure

• Bunch unpackaged unindexed
• Set packaged unindexed

• String unpackaged indexed
• List packaged indexed 24 October 2026 35

Slide 36 — LIST Theory 24 October 2026

(No body text on this slide.)

Slide 37 — LIST THEORY

• Alist is a string in a package, just like a set is a bunch in a package.
[0; 1; 2] list of three items [0; 1; 2]: [nat; 1; (0,..10)]
On the left side of the colon is a list of integers, on the right is a list of bunches.
• List brackets [ ] distributes over bunches
[null] = null [A, B] = [A], [B]
24 October 2026 37

Slide 38 — List Theory

SYNTAX Let S be a string, L and M are lists, n:nat and i be an item.
• [S] “list containing S ” ~L “contents of L ”, is a string ~ [3; 5; 7; 4] = 3; 5; 7; 4
• L M “SubList of [L], that is composed from the items indexed by ~M in L “
• Example: [3; 5; 7; 4] [2; 1; 2] = [7; 5; 7]
• L;; M “ L join M ” join TWO LISTS
• Example: [3; 5; 7; 4];; [2; 1; 2] = [3; 5; 7; 4; 2; 1; 2]
24 October 2026 38

Slide 39 — List Theory SYNTAX

S is a string, L and M are lists, n:nat and i is an item.
n→i | L “ n maps to i otherwise L ”
Store in index n the item i, the rest stay as it was
Example 1:
• 2→53 | [10;..15] = 2→53 | [10;11;;13;14;15] = [10; 11; 53; 13; 14]
• 2→53 | 3→65 | [10;..15] = [10; 11; 53; 65; 14]
24 October 2026 39

Slide 40 — List Theory SYNTAX

S is a string, L and M are lists, n:nat and i is an item.
n→ i | L “ n maps to i otherwise L ”
Store in index n the item i, the rest stay as it was
Example 2:
Let L = [10; 6; 8; 9; 15] 2→ L 3 | 3→ L 2 | L = 2→ L 3 | 3→ 8| L = 2→ L 3 | [10; 6; 8; 8; 15]
= 2→ 8 | [10; 6; 8; 8; 15] = [10; 6; 8; 8; 14]
24 October 2026 40

Slide 41 — List Theory SYNTAX

S is a string, L and M are lists, n:nat and i is an item.
n→ i | L “ n maps to i otherwise L ”
Example:
Let L = [10;..15] = [10; 11; 12; 13; 14]
2→ L 3 | 3→ L 2 | L = [10; 11; 13; 12; 14]
The point is that all 3 Ls in that expression i.e., L L2 and L3, refer to
L, the original list.
This form meant to exchange elements 2 and 3 in the list 24 October 2026 41

Slide 42 — List Theory SYNTAX

• Sis a string, L and M are lists, n: nat; and i is an item
#L “length of L ” is xnat, #[3; 5; 7] = 3
 L “domain of L ” is a bunch of natural numbers
 [3; 5; 7; 4] = 0,..4 (range of the list indexes)
• L n “ L at index n ” is an item [3; 5; 7; 4] 2 = 7 24 October 2026 42

Slide 43

L and M are lists, S and T are strings, n be an index of S, i is an item, and A and B be bunches of strings.
L== [S] S structure [~L] = L list formation
~[S] = S contents #[S] = ↔ S List length = #[S] == length of its contents/string == ↔ ~[S]
 L = 0,..#L list domain (index range)
[S];;[T] = [S; T] join
[S] n = Sn indexing [S] [T] = [S] List composition T n→ i | [S] = [S n i] modification
[S] = [T] ≡ S = T equation
[S] < [T] = S < T order [A]: [B] = A: B inclusion 24 October 2026 43

Slide 44

List Theory AXIOMS Indexing with any complicated structure, results in that same structure String List
S = null L null = null null
S = S, S L (A, B) = L A, L B
A,B A B S = {S } L {A} = {L A}
{A} A
S = nil L nil = nil nil
S = S; S L (S; T) = L S; L T
T;U T U S = [S] L [S] = [L S]
[T] T

Slide 45 — List Theory EXAMPLES

• Let S = 10; 11; 12 Then
• S = S0, {S1, [S2; S1]; S0} = 10, {11, [12; 11]; 10}
0, {1, [2; 1]; 0}
• Let L =[S] = [10; 11; 12] Then L (0, {1, [2; 1]; 0})
= L 0, {L 1, [L 2; L 1]; L 0}
= 10, {11, [12; 11]; 10} a bunch with mixed structures
A= 0,2 B= 0,1 Then L (A, B) = L (0,2), L(0,1)
L (A, B) = 10, 12, 10,11= 10, 11, 12 a bunch 24 October 2026 45

Slide 46 — Multidimensional Structures Lists

• Atwo -dimensional list is a list whose items are lists.
• It is used to be called an array, but now the word array represents any number of dimensions.
• let A = [[6; 3; 7; 0]; [4; 9; 2; 5]; [1; 5; 8; 3]]
Then A is a 2-dimensional, or a 3× 4 array.
A: [3*[4*nat]]. A 1 = [4; 9; 2; 5] A 1 3 = 5 A 2 0 = 1
Example:
A (1, 2) = A 1, A 2 = [4; 9; 2; 5], [1; 5; 8; 3]
24 October 2026 46

Slide 47 — Multidimensional Structures Lists

• Atwo -dimensional list is a list whose items are lists.
• It is used to be called an array, but now the word array represents any number of dimensions.

A = [[6; 3; 7; 0]; [4; 9; 2; 5]; [1; 5; 8; 3]]
Example 2:
A [1, 2] = [A 1, A 2] = [[4; 9; 2; 5], [1; 5; 8; 3]]
= [[4; 9; 2; 5]], [[1; 5; 8; 3]] = B B:A 24 October 2026 47

Slide 48 — DO ASSIGNMENT THREE 24 October 2026 48

(No body text on this slide.)

Slide 49 — FUNCTION THEORY 24 October 2026

(No body text on this slide.)

Slide 50 — int succ (int v) { return v + 1; } C++ f…

 v: d→ v+ 1
 function in our theory of programming int succ (int v)
{ return v + 1; } int main()
{ int count = succ (7);
std:: cout << "The sum is: " << sum << std:: endl;
return 0;
s ucc 7 =
 v: d→ v+ 1 7 = 7 + 1 = … The rest depends on the syntax of the programming language 24 October 2026 50

Slide 51 — FUNCTION Y = F x where x: D

Y= f x where x:D
Note: you may use F(x), but the Parentheses are reserved for other purposes
Y= x+ 1 where x:nat define and generalize
 x: D expression of the function (body)
 Y=   The brackets   indicate the scope of the variable “x” and its domain axiom.
y = x: D x+ 1 == v: D v+ 1
 illustrate y =
 x: D→ body name =
 v: D→ b
 for arbitrary function succ =
 v: nat→ v+ 1
 substitute parameter value to get the range succ 4 =
 v: nat→ v+ 1
 4 = 4 + 1 = 5 Function application 24 October 2026 51

Slide 52

• Afunction introduces a variable ‘v’ whose purpose is to help express the mapping from domain elements to range elements. v: D→ b
• The variable name should be fresh, not already in use for another purpose.
• The Renaming Axiom:
if v and w are names, and neither v nor w appears in D, and w does not appear in b, then
 v: D→ b = w: D→ (substitute w for v in b)
succ = v: nat→ v+ 1 == w: D→ w+ 1
• Formalization of the function name = v: D→ b
• Application of the function f x “ f applied to x ” or “ f of x ” is the corresponding element of the range. F x = v: D → b x 24 October 2026 52

Slide 53 — FUNCTION THEORY

• f(n) = n+ 1, n ∈ D (some domain)
• □ v: D→ b □ “map v in D to b ”, “local v in D maps to b ”
Body (can use v)
Domain, type(bunch)
Variable, parameter
• □ v: nat↦ v+ 1 □ 24 October 2026

Slide 54 — FUNCTION THEORY

• □ f is the domain of f.
• The Domain Axiom:
□ □ v: D↦ b □ = D □ □ n: nat n+ 1□ = nat
• Dis the domain of function □ v: D→ b□ and within the body b, D is the domain of variable v.
#f = ¢ □ f is the size of function f The range of a function consists of the elements obtained by substituting each element of the domain for the variable in the body.
24 October 2026

Slide 55 — FUNCTION THEORY

• Renaming
□ m: nat↦ m +1 □ □ v: nat↦ v +1 □
• The Domain Axiom:
□ f is the domain of f. □ □ v: nat nat +1 □ = nat #f = ¢ nat =∞
• Application Axiom. If element x: D, then
□ v: D↦ b □ x = (substitute x for v in b)
Application F x □ m: nat↦ m +1 □ x ≡ x+1 suc = □ n: nat↦ n+1 □ , suc 3 = □ n: nat↦ n+1 □ 3 = 3+1 = 4 24 October 2026

Slide 56 — FUNCTION THEORY

• Two Variables/Parameters avg = □ x: rat→ □ y: rat→ (x+y)/2 □ □
avg 3 = □ y: rat→ (3 +y)/ 2 □ avg 3 5 = (3 + 5)/2 = 4 Do not use parenthesis, because avg(3,5) == avg(3), avg(5) is a Bunch of avg 24 October 2026

Slide 57 — FUNCTION THEORY

• Apredicate is a function whose body is a binary expression.
even = □ i: int→ i/2: int□ odd = □ i: int↦ ¬ i/2: int□
• Arelation is a function whose body is a predicate.
divides = □ n: nat+1→ □ i: int→ i/n: int□ □
divides 2 = □ i: int→ i/2: int□ = even divides 2 8 = □ 2: nat+1↦ □ 8: int→ 8/2: int□ □
= 8/2: int = 4:int = T 24 October 2026

Slide 58

Variable Scope and Substitution in the function (exactly as that in the programming Language)
• Local variable bound, hidden, private introduced inside the expression (formal)
• Nonlocal variable global, free, visible, public its introduction is outside the expression (whether formal or informal).
□ x→ x y □ (x y)
24 October 2026

Slide 59 — Body of the Rectifying the Local variabl…

outer 1. R= □ x:D→ x □ x:D→ x □ x □ 4 RECTIFY function
R== □ x:D→ x □ z:D→ z □ x □ 4 == (4 □ ≠ z □ 4)
2. S== □ y→ y x□ x→ x y □ y x □ x state nonlocals and locals var in outer function
== □ y→ y x□ x→ x y □ y x □ x subst x/argumant for each occurrence of y
== (x x□ x→ x x□ x x) the underlined x is nonlocal, yet reintroduces as local in the inner function, must Rectify before subst
S == □ y→ y x□ z→ z y □ y x □ x subst x for each occurrence of y == (x x□ ≠ z x□ x x)
FATMA A. EL

Slide 60 — Examples 24 October 2026

(No body text on this slide.)

Slide 61 — Rectify and evaluate

□ y→ x y □ x→ x y □ x y □ x renaming inner local x
= □ y→ x y □ z→ z y □ x y □ x substitution x for each occurrence of y
= (x x □ z→ z x □ x x) sub. x for each occurrence of z = x x x x x 24 October 2026

Slide 62 — INSTANTIATE THEN SIMPLIFY

• Given the expression p (x+y) (2×u + w) z (x, y, u, w, z: int)
Replace p by
□ x: int→ □ y: int→ □ z: int→ x≥ 0 ∧ x^2 ≤ y ∧ □ z: int· z^2≤ y⇒ z≤ x□ □ □
Simplify
§ □ x: int→ □ y: int→ □ z: int→ x≥ 0 ∧ x^2≤ y ∧ □ z: int· z^2≤ y⇒ z ≤ x□ □ □
(x+y) (2×u + w) z
• Variables x, y, and z appear both locally and nonlocally.
• Variable z is introduced, locally, twice.
• Rectify: To avoid confusion, rename the local variables to a, b, c, and d.
24 October 2026 62

Slide 63

§ □ x: int→ □ y: int→ □ z: int→ x≥0 ∧ x^2≤y ∧ □ z: int· z^2≤y⇒ z ≤ x□ □ □ (x+ y) (2×u + w) z ≡ □ a: int→ □ b: int → □ c: int→ a≥0 ∧ a^2 ≤b ∧ □ d: int· d^2≤ b⇒ d ≤ a□ □ □ (x+ y) (2×u + w) z ≡ x+y ≥ 0∧ (x+y)^2 ≤ 2×u+w ∧ □ d: int· d^2 ≤ 2×u+ w⇒ d ≤ x+ y
what about z?!!
Assignment: Assume values for x, y, u, w that satisfies this expression (evaluate it to T)
24 October 2026

Slide 64 — Quantifiers 24 October 2026

(No body text on this slide.)

Slide 65 — QUANTIFIERS

• Aquantifier is a one-operand prefix operator that applies to functions.
Considered Quantifiers are □ , Σ,∏, ∃, ⇔, ⇐
Any two-operand symmetric associative operator can be used to define a quantifier.
• □ defined by∧
a ∧ b == b ∧ a A ∧ (B ∧ C) == (A ∧ B) ∧ C Ex.
□ □ r: rat→ r ≥ 0 □ =⊥ “for all r in rat...”
• ∃ defined by ∨
Ex.

∃ □ n: nat→ n= 0 □ =⊤ “there exists n in nat such that...”
24 October 2026

Slide 66 — QUANTIFIERS

• Σ defined by +
n Ex. Σ
□ n: nat+1→ 1 /2
□ = 1 the sum, for n in nat+1, of...”
• Π defined by ×
Ex. Π
□ n: nat+1→ n/(n+1)
□ = 0 “the product, for n in nat+1, of...”
• □ defined by ↑
Ex.
□ □ y: rat→ y× (4 –y)
□ = 4 “the maximum(lub), for x in rat, of...”
• □ defined by ↓
Ex.
□ □ n: nat+1→ 1 /n
□ = 0 “the minimum (glb), for n in nat+1, of...”
24 October 2026

Slide 67 — QUANTIFIERS

• Abbreviations
• □ r: rat· r≥ 0 abbreviates □ r: rat→ r ≥ 0 □
n n
• Σn: nat+1 · 1 /2 abbreviates □ n: nat+1→ 1 /2 □
• □ x, y: rat· x = y+1⇒ x > y abbreviates □ x: rat· □ y: rat· x = y+1⇒ x > y
• Σn, m: 0,..10 · n× m abbreviates Σn: 0,..10 · Σm: 0,..10 · n× m 24 October 2026

Slide 68 — QUANTIFIERS Axioms

• v is a name, A and B are bunches, b is a binary expression, n is a number expression, and x is an element. (∧ and ∨ are idempotence)
□ v: null· b =⊤ ∃ v: null· b = ⊥
□ v: x· b = □ v: x↛ b □ x∃ v: x· b = □ v: x↛ b □ x
□ v: A, B· b = (□ v: A· b)∧ (□ v: B· b)∃ v: A, B· b =(∃ v: A· b) ∨ (∃ v: B· b)
Σ v: null· n = 0 Π v: null· n = 1
Σ v: x· n = □ v: x↛ n□ x Π v: x· n = □ v: x↛ n□ x
□ v: null· n =∞ □ v: null· n = –∞
□ v: x· n = □ v: x↛ n□ x □ v: x· n = □ v: x↛ n□ x
□ v: A, B· n = (□ v: A· n) ↓ (□ v: B· n) □ v: A, B· n = (□ v: A· n) ↑ (□ v: B· n)
24 October 2026

Slide 69 — QUANTIFIERS Axioms over Null

• v is a name, A and B are bunches, b is a binary expression, n is a number expression, and x is an element.
Null ∞
• □ v: null· b =⊤ ∃ v: null· b = ⊥
… … 1 OOO …
X+ 0 x X1
Σ v: null· n = 0 Π v: null· n = 1
-1 …
-10
□ v: null· n =∞ □ v: null· n = –∞

-1000 … …
GLB of the null is ∞ LUB of the null is -∞
-∞ Null 24 October 2026

Slide 70

QUANTIFIERS Axioms over an element n is a number expression, and x is an element
□ v: x· b = □ v: x↛ b □ x □ v: x· v>8 = □ v: x↛ v>8 □ x == x>8
∃ v: x· b = □ v: x↛ b □ x∃ v: x· v<5 = □ v: x↛ v<5 □ x == x<5 2 2
Σ v: x· n = □ v: x↛n □ x Σ v: 5· v+4 = 5 +4 = 29
Π v: x· n = □ v: x↛n □ x Π v: 3· v+1 = 3+1 = 4
□ v: x· n = □ v: x↛ n□ x □ v: 14· v +3 = 14+3 = 17
□ v: x· n = □ v: x↛ n□ x □ v: 12· v+ v = 24 24 October 2026

Slide 71

QUANTIFIERS Axioms Over Bunch Union Let A= 2, 9 B= 2, 8 n is a number expression (∧ and ∨ are idempot ance)
• □ v: A, B· b = (□ v: A· b)∧ (□ v: B· b)
2 2 2 Ex. □ v: A, B· v -6 > 0 == □ v: A· v -6 > 0∧ □ v: B· v -6 > 0
==(4 -6 > 0) ∧ (81 -6) > 0)∧ (4 -6 > 0) ∧ (64 -6) > 0 == ⊥
• ∃ v: A, B· b =(∃ v: A· b) ∨ (∃ v: B· b)
2 2 2 Ex. ∃ v: A, B· v -3 > 0 == ∃ v: A· v -3 > 0 ∨ ∃ v: B· v -3 > 0
==(4 -3 > 0) ∨ (81 -3) >0)∨ (4 -3 > 0) ∨ (64 -3) >0
== ⊥ ∨ T ∨ ⊥ ∨ T
== T 24 October 2026

Slide 72

QUANTIFIERS Axioms Over Bunch Union Let A= 2, 9 B= 2, 8 and n is a numeric expression
Σ and Π are not idempotence
• (Σ v: A, B· n) + (Σ v: A‘B· n) = (Σ v: A· n) + (Σ v: B· n)
2 2
LHS=(Σ v: A, B· v + 3) + (Σ v: A’B· v + 3) = (4 + 3 + 81 + 3 + 64 + 3) + (4+ 3)= 165 2 2
RHS=(Σ v: A. v + 3) + (Σ v: B· v + 3) = (4 + 3 + 81 + 3) + (4+ 3 + 64 + 3) =
= 91 + 74 = 165
• (Π v: A, B· n) × (Π v: A‘B· n) = (Π v: A· n) × (Π v: B· n)
2 2 2 2
LHS = (Π v: A, B· v + 3) × (Π v: A‘B· v + 3) = Π v: 2, 9, 8. v + 3 × (Π v: 2· v + 3)
= (4+ 3) × (81 + 3) × (64 + 3) × (4+ 3) = 7 × 84 × 67 × 7
RHS = ((4+ 3) × (81 + 3)) × ((4+ 3) × 64 + 3) = 7 × 84 × 7 × 67 24 October 2026

Slide 73 — THE SOLUTION QUANTIfiER §

• The solution quantifier “§” applies to predicates and gives its solutions.
• (“solution(s) of”, “those”, “that” (singular)) gives the bunch of solutions of a predicate.
• the solution axioms.
§v: null· b = null
§v: x· b = if □ v: x↛ b□ x then x else null fi Ex §v: x· v> 5 = if x> 5 then x else null fi 24 October 2026

Slide 74 — THE SOLUTION QUANTIfiER §

• The solution quantifier “§” applies to predicates.
(“solution(s) of”, “those” (plural), “that” (singular)) gives the bunch of solutions of a predicate.
Examples:
§i: int· i= 4 == –2, 2 “those i in int such that...”
§n: nat· n<3 = 0,..3

{§n: nat· n<3} ≡ {n: nat· n<3} ={0,..3}
{§i: int· i= 4 } ≡ {-2, 2} 24 October 2026

Slide 75 — READ AND STUDY

• There are further axioms to say how each quantifier behaves when the domain is a result of the § quantifier; they are listed in the handout, together with other laws concerning quantification. These laws are used again and again during programming;
they must be studied until they are all familiar 24 October 2026

Slide 76 — Law Illustration

• Generalization Laws
• if p is a predicate and element x: p,
 x: p. p x⇒ p x ⇒ ∃ x: p. p x
• If p results in ⊤ for all its domain elements, then p results in ⊤ for domain element x.
• And if p results in ⊤ for domain element x, then there is a domain element for which p results in ⊤.
• Similarly, if f is a numeric function and element x: f
• f ≤ f x ≤ f 24 October 2026

Slide 77 — Law Illustration

• If p is a predicate that does not mention nonlocal variable x (mention only the local variable x), and element y is in the domain of p (y: p), then x: p· x=y⇒ p x = ∃ x: p· x=y∧ p x == p y This is what is called One Point Law (very useful for minimizing quantified expression into a classified point in the given domain)
As it maps a quantified expression to a point in the predicate Domain.
This one point in the predicate domain can then be classified 24 October 2026

Slide 78

You Can Now, Finish Assignment Three And start in Assignment Four 24 October 2026